6 - 部署上线

本节重点

学习 Docker 容器化和云端部署的完整流程，实现项目的生产环境部署，让 AI 自动回复工具真正运行起来。

具体内容包括：

- Docker 容器化
- Railway 一键部署
- 日志监控

建议大家在学习过程中多动手实践，真正把项目部署到云端运行起来，这样才能体验到完整的项目流程，不能只会在本地运行应用。

💡 鱼皮在 编程导航 带大家做过十几套项目了，几乎每种部署方式都给大家讲解过，直接看对应项目的部署上线视频教程即可。

- AI 零代码应用生成平台项目：1Panel 面板 + Nginx 前端 + Java 后端（jar 包）
- 代码生成器共享平台项目：宝塔面板 + Nginx 前端 + Java 项目管理器（jar 包）后端部署
- AI 答题应用平台项目： Vercel 前端 + Docker 后端 + 云托管 Serverless 平台部署
- AI 超级智能体项目：Docker 前端 + Docker 后端 + 云托管 Serverless 平台部署
- OJ 在线判题项目：Docker Compose 后端微服务部署

基本上学会这几种部署方式，能够应对绝大多数部署需求了。

一、Docker 容器化

Docker 是主流的、开源的容器技术，能够将应用程序及其依赖环境打包成轻量级、可移植的容器，实现 “一次构建，到处运行” 的部署模式。很适合快速准备开发环境和部署项目。

编写 Dockerfile

如果想基于 Docker 部署项目，需要编写 Dockerfile 文件，这是一个包含构建指令的文本文件，定义了如何从基础镜像开始逐步构建自定义 Docker 镜像的完整步骤和配置。

项目的 Dockerfile 内容如下，一般去开源平台参考别人项目的写法修改即可：

▼

dockerfile
复制代码

```
FROM maven:3.8.1-jdk-8-slim as builder

MAINTAINER yupi

# Copy local code to the container image.
WORKDIR /app
COPY pom.xml .
COPY src ./src

# Build a release artifact.
RUN mvn package -DskipTests

# 声明环境变量，这样容器就可以在运行时访问它们
ENV OPENAI_MODEL=text-davinci-003
ENV OPENAI_API_KEY=你的API_KEY
ENV ZSXQ_COOKIE=你的星球Cookie
ENV ZSXQ_GROUP_ID=你的星球id
# 是否只提醒提问者
ENV ZSXQ_SILENCED=true

# Run the web service on container startup.
ENTRYPOINT ["java","-jar","/app/target/yu-auto-reply-0.0.1-SNAPSHOT.jar","--spring.profiles.active=prod"]
```

注意，我们要以 prod 配置启动项目，因此需要编写一个 `application-prod.yml` 文件：

▼

yaml
复制代码

```
spring:
  application:
    name: yu-auto-reply
  mvc:
    pathmatch:
      matching-strategy: ant_path_matcher
server:
  address: 0.0.0.0
  port: 8080
  servlet:
    context-path: /api

# openAI 配置
openai:
  model: ${OPENAI_MODEL:text-davinci-003}
  apiKey: ${OPENAI_API_KEY:你的apiKey}

# 知识星球配置
zsxq:
  cookie: ${ZSXQ_COOKIE:你的星球cookie}
  groupId: ${ZSXQ_GROUP_ID:你的星球id}
  silenced: ${ZSXQ_SILENCED:true}

# 任务配置
task:
  # 并发
  concurrent:
    # 默认关闭（串行）
    enable: false
    # 并发大小（不填则等同于任务数，即全并发）
    size: 1
  # 任务列表，支持配置多个
  list:
    - name: task1 #任务名
      monitor: zsxq #监控者
```

```
    answerer: openai #回答者
    cron: '0/30 * * * * ?' #执行周期
```

💡 如果项目部署依赖的组件较多，还可以利用 Docker Compose 编排 Docker 容器的群起，简化部署。比如下面这个示例 Docker Compose 配置：

▼
yaml
复制代码
```
version: '3.8'

services:
  yu-auto-reply:
    build: .
    ports:
      - "8080:8080"
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ZSXQ_COOKIE=${ZSXQ_COOKIE}
      - ZSXQ_GROUP_ID=${ZSXQ_GROUP_ID}
      - ZSXQ_SILENCED=${ZSXQ_SILENCED:-true}
    restart: unless-stopped
```

环境变量配置

创建 `.env` 文件，用于给项目传递配置（不要提交到版本控制）：

▼
plain
复制代码
```
# OpenAI 配置
OPENAI_API_KEY=your_openai_api_key_here
OPENAI_MODEL=text-davinci-003

# 知识星球配置
ZSXQ_COOKIE=your_zsxq_cookie_here
ZSXQ_GROUP_ID=your_group_id_here
ZSXQ_SILENCED=true

# 其他配置
SPRING_PROFILES_ACTIVE=prod
```

构建和运行

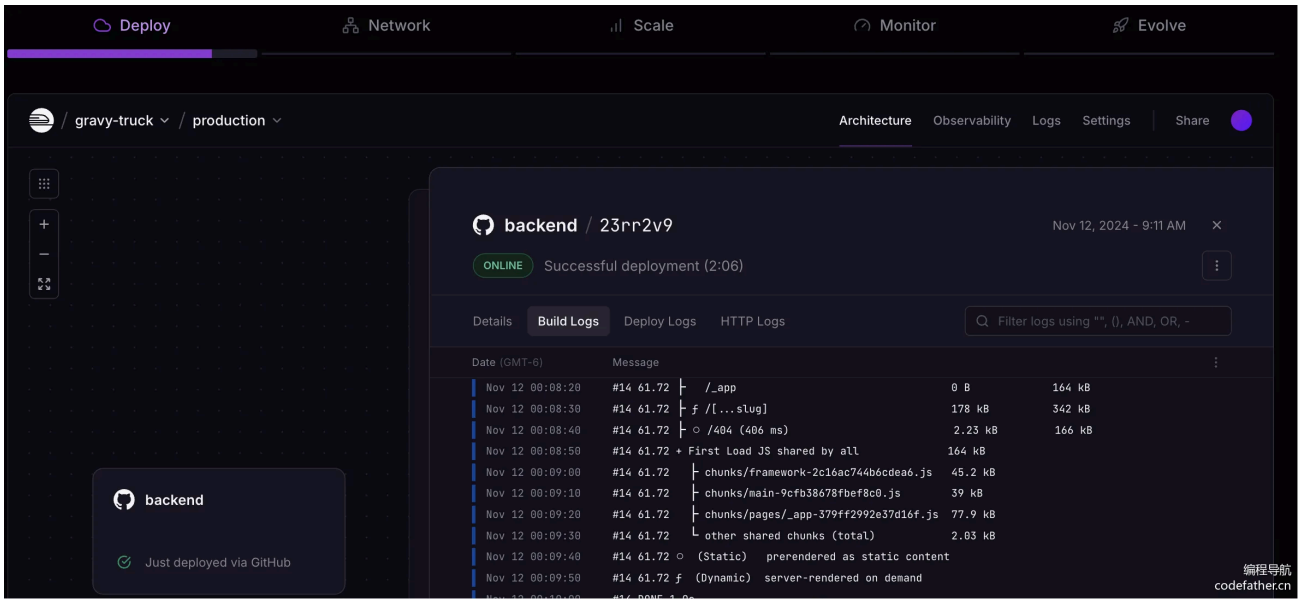
通过 docker 命令可以构建镜像和运行容器，运行容器时可以额外指定一些环境变量。

▼
bash
复制代码
```
# 构建 Docker 镜像（会自动构建项目）
docker build -t yu-auto-reply:latest .

# 运行容器
docker run -d \
  -p 8080:8080 \
  -e OPENAI_API_KEY=your_api_key \
  -e ZSXQ_COOKIE=your_cookie \
  -e ZSXQ_GROUP_ID=your_group_id \
  yu-auto-reply:latest

# 查看日志
docker logs -f <container_id>

# 停止容器
docker stop <container_id>
```

二、Railway 一键部署

Railway 是一个现代化的云平台，专为开发者设计，提供简单易用的应用部署和管理服务。它的优势包括：

1. 零配置部署：自动检测项目类型并配置构建环境
2. Git 集成：支持从 GitHub 直接部署
3. 环境变量管理：提供安全的配置管理
4. 自动扩展：根据负载自动调整资源
5. 实时日志：提供实时的应用日志查看



准备部署文件

Railway 平台支持自动检测项目类型，对于 Spring Boot 项目，通常不需要额外的配置文件。Railway 会自动：

1. 检测到这是一个 Maven 项目
2. 运行 `mvn clean package -DskipTests` 构建项目
3. 启动生成的 jar 文件

如果需要自定义构建过程，可以创建以下配置文件：

▼
```
json
复制代码
{
  "$schema": "https://railway.app/railway.schema.json",
  "build": {
    "builder": "NIXPACKS"
  },
  "deploy": {
    "startCommand": "java -jar target/yu-auto-reply-0.0.1-SNAPSHOT.jar"
  }
}
```

部署步骤

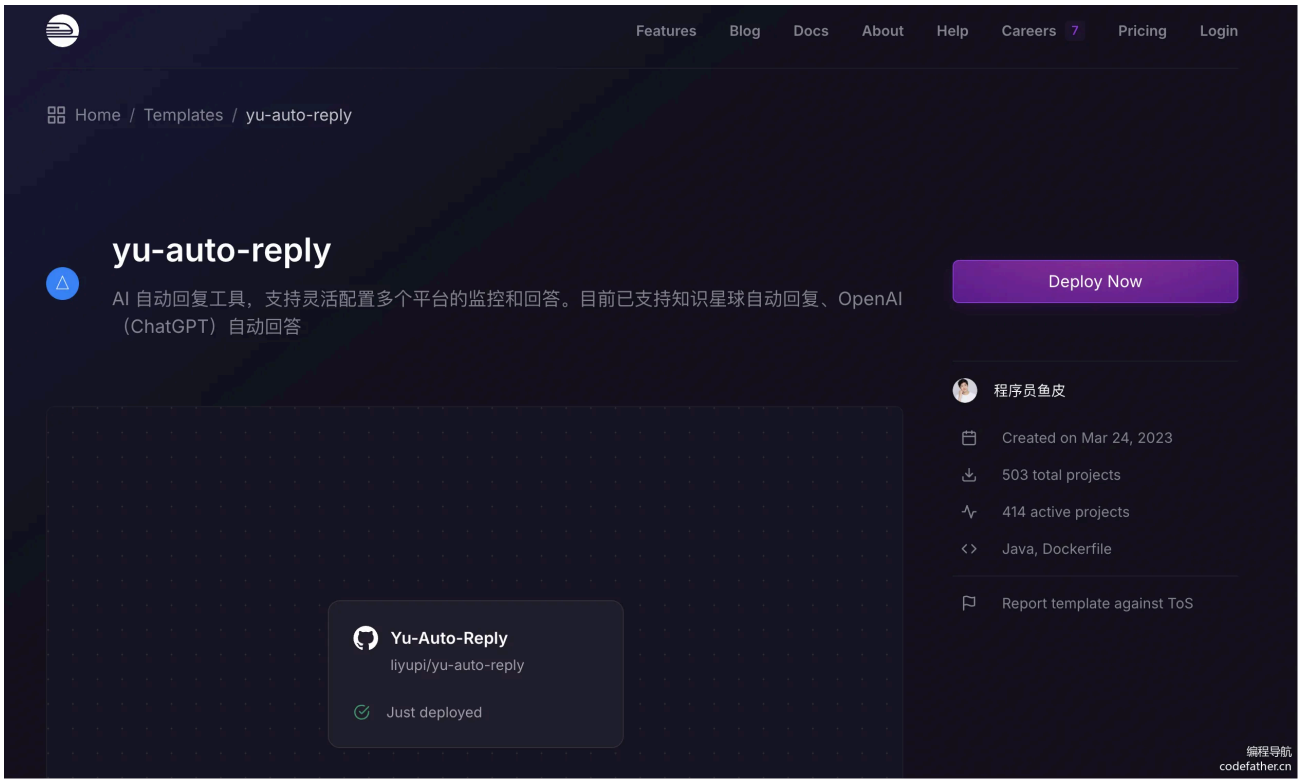
1、准备 GitHub 仓库

▼
```
bash
```

复制代码
```
# 初始化 Git 仓库
git init

# 添加 .gitignore
echo "target/" >> .gitignore
echo "*.log" >> .gitignore
echo ".env" >> .gitignore

# 提交代码
git add .
git commit -m "Initial commit"

# 推送到 GitHub
git remote add origin https://github.com/yourusername/yu-auto-reply.git
git push -u origin main
```

2、Railway 部署

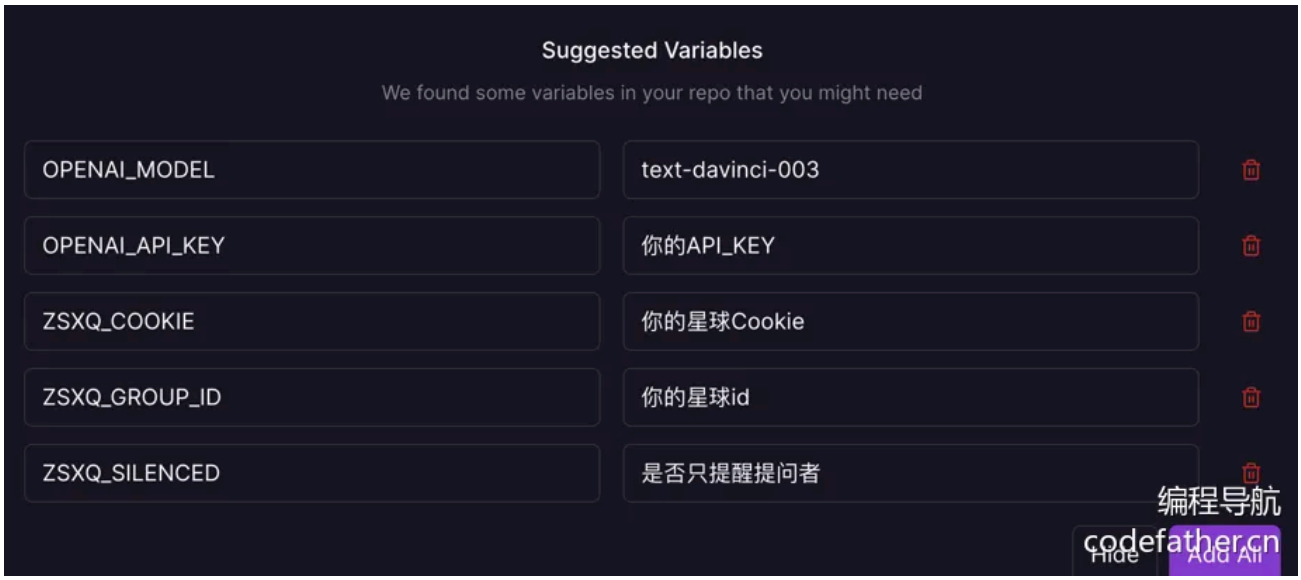
进入 Railway，新建项目并选择 GitHub 仓库，接下来就是傻瓜式操作了~



3、配置环境变量

在 Railway 控制台的 Variables 标签页中添加：
▼
plain
复制代码
```
OPENAI_API_KEY=your_openai_api_key
OPENAI_MODEL=text-davinci-003
ZSXQ_COOKIE=your_zsxq_cookie
ZSXQ_GROUP_ID=your_group_id
ZSXQ_SILENCED=true
```

如图：



💡 Railway 平台会自动为应用分配合适的资源，但可以通过环境变量优化 JVM 参数：

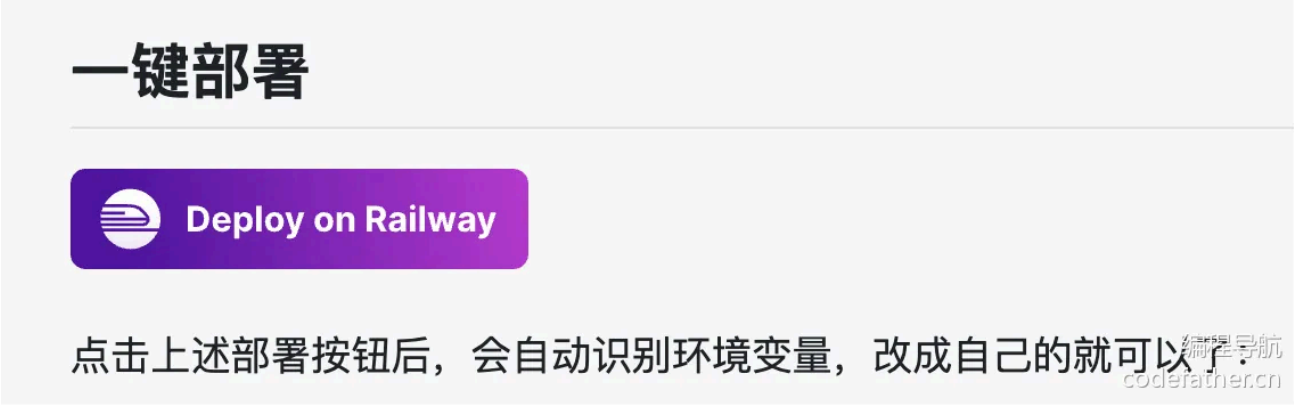
▼
bash
复制代码
```
# 在 Railway 中设置环境变量
JAVA_OPTS=-Xms128m -Xmx256m
```

4、自定义域名（可选）

在 Railway 控制台的 Settings 标签页中，可以配置自定义域名。

5、提供部署按钮

你还可以像鱼皮一样，给开源项目 README 补充一键部署项目到 Railway 的按钮，本质上就是给文档补了个超链接，但是看起来很有逼格，可以自己尝试一下。



四、日志监控

项目使用 Spring Boot 默认的日志配置，主要通过以下方式记录日志：

1、启动日志

MainApplication 启动时会记录配置信息：

▼
java
复制代码

```
@Override
public void run(String... args) throws Exception {
    OpenAiConfig openAiConfig = SpringContextUtils.getBean(OpenAiConfig.class);
    ZsxqConfig zsxqConfig = SpringContextUtils.getBean(ZsxqConfig.class);
    log.info("OpenAi 配置 {}", openAiConfig);
    log.info("知识星球配置 {}", zsxqConfig);
}
```

2、任务执行日志

TaskConfig 在任务注册和执行时会记录相关日志：

▼

```
java
复制代码
log.info("--- 任务注册开始 ---");
// ... 任务注册逻辑
log.info("任务注册成功 {}", taskListItem);
log.info("--- 任务注册结果 ---");
```

3、监控日志

ZsxqMonitor 在执行监控时会记录详细日志：

▼

```
java
复制代码
log.info("任务 {} 监控开始", taskName);
log.info("{} 收到新提问 \n 问题：{}", taskName, question);
log.info("{} 回答成功 \n 问题：{} \n 回答：{}", taskName, question, answer);
log.error("{} 回答失败 \n 问题：{}", taskName, question);
log.info("任务 {} 监控结束", taskName);
```

4、异常日志

项目中的全局异常处理器会记录异常信息：

▼

```
java
复制代码
@ExceptionHandler(BusinessException.class)
public BaseResponse<?> businessExceptionHandler(BusinessException e) {
    log.error("BusinessException", e);
    return ResultUtils.error(e.getCode(), e.getMessage());
}

@ExceptionHandler(RuntimeException.class)
public BaseResponse<?> runtimeExceptionHandler(RuntimeException e) {
    log.error("RuntimeException", e);
    return ResultUtils.error(ErrorCode.SYSTEM_ERROR, "系统错误");
}
```

目前有这些日志信息就足够了，随着项目的逐渐扩展，可以给项目增加企业级的监控系统，推荐学习 鱼皮 AI 零代码应用生成平台项目 的第 13 期 —— 可观测性。

总结

至此，本项目教程就完结啦！这个项目虽然实现并不复杂，但也涉及到了企业级开发的方方面面：

- 架构设计：运用设计模式实现高内聚、低耦合的系统架构
- 第三方集成：学会了如何对接 OpenAI、知识星球等外部 API
- 任务调度：掌握了 Spring Boot 定时任务的高级用法
- 容器化部署：学会了 Docker 和云平台部署的完整流程

希望大家能够举一反三，将这些经验应用到其他项目中去，加油！